📸 Snapchat-style Photo Filters with OpenCV

Face detection + fun overlay filters + a live-webcam version

You already know: variables, data types, lists/dicts/tuples/sets, operators, `if-elif-else`, `for` / `while` loops, functions, and the OpenCV basics (images = NumPy arrays, BGR colour, drawing, ROIs).

🎯 What we will build

1. A **face + eye detector** using OpenCV's Haar cascades.
2. **Colour filters** (sepia, vintage, cool/warm, vignette, cartoon, colour-pop) — the "lens filters".
3. **Sticker filters** (dog ears + nose, sunglasses, moustache, bunny ears, crown) — built as **transparent (RGBA) stickers** and glued onto the face, exactly like real filter apps do with PNG assets, except we *draw* our stickers with OpenCV instead of downloading images.
4. A **big-eyes cartoon filter** using local image warping.
5. A single `apply_filter(image, name)` function that ties everything together — a mini "filter engine".
6. A **live webcam application**: real-time face detection, filters you switch with the keyboard, an FPS counter, and snapshot saving — the script to actually run on your own laptop.

⚠️ How Snapchat *really* does it (and why ours is simpler)

Real Snapchat/Instagram filters use a **face-landmark model** (dozens of points: eye corners, nose tip, jaw, lips) built with deep learning (their own tech, or open tools like **dlib** / **MediaPipe** / **face-alignment**), so a filter is glued to the *exact* eye or nose pixel. This notebook's environment does not have those libraries installed, so we use the **classic OpenCV toolbox**: Haar-cascade face/eye **boxes**, and we place stickers using **proportions of that box** (e.g. "the nose is about 55% down the face, centred"). It is less precise (filters can drift a little if you tilt your head) but it teaches the same core ideas, runs anywhere, and is a great base to upgrade later with `pip install mediapipe`.

0️⃣ Setup

In [ ]:
```
# pip install opencv-python numpy matplotlib     # <- uncomment if needed
```

In [ ]:
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
import os

print("OpenCV version:", cv2.__version__)
print("NumPy version :", np.__version__)
```

In [ ]:
```
# ---- Helper functions (we will learn how to write functions later; for now jus

def show(img, title=""):
    """Show ONE image. Works for both grayscale and colour (BGR) images."""
    plt.figure(figsize=(6, 4))
    if len(img.shape) == 2:                          # grayscale -> only (height, w
        plt.imshow(img, cmap="gray", vmin=0, vmax=255)
    else:                                            # colour -> convert BGR to RGB
        plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
    plt.title(title)
    plt.axis("off")
    plt.show()


def show_many(images, titles, cols=3):
    """Show several images side by side. images and titles are two lists of the
    rows = (len(images) + cols - 1) // cols
    plt.figure(figsize=(4 * cols, 3.3 * rows))
    for i in range(len(images)):
        plt.subplot(rows, cols, i + 1)
        if len(images[i].shape) == 2:
            plt.imshow(images[i], cmap="gray", vmin=0, vmax=255)
        else:
            plt.imshow(cv2.cvtColor(images[i], cv2.COLOR_BGR2RGB))
        plt.title(titles[i])
        plt.axis("off")
    plt.tight_layout()
    plt.show()
```

1️⃣ Get a face to work with

Two ways to run this notebook:

- **Best experience:** put a clear, front-facing selfie next to this notebook named `face.jpg`.
- **No photo? No problem:** we draw a simple **cartoon face** with OpenCV shapes so every cell below still runs. Haar cascades are trained on *real* faces, so they will **not** detect the cartoon one — when that happens we fall back to a manually-defined face box just so you can *see* every filter in action. The code path (`detectMultiScale`, etc.) is 100% the same one used on a real photo.

In [ ]:
```
USING_REAL_PHOTO = os.path.exists("face.jpg")

if USING_REAL_PHOTO:
    photo = cv2.imread("face.jpg")
    print("Loaded your own photo: face.jpg", photo.shape)
else:
    photo = np.full((480, 480, 3), 245, np.uint8)
    cv2.ellipse(photo, (240, 250), (130, 165), 0, 0, 360, (200, 220, 255), -1)
```

```
    cv2.ellipse(photo, (240, 420), (90, 60), 0, 0, 180, (200, 220, 255), -1)
    cv2.circle(photo, (190, 215), 16, (30, 30, 30), -1)
    cv2.circle(photo, (290, 215), 16, (30, 30, 30), -1)
    cv2.ellipse(photo, (240, 260), (12, 22), 0, 0, 360, (150, 170, 210), -1)
    cv2.ellipse(photo, (240, 320), (42, 18), 0, 0, 180, (60, 30, 80), 3)
    cv2.ellipse(photo, (240, 60), (140, 90), 0, 180, 360, (40, 60, 90), -1)
    print("face.jpg not found -> using a drawn cartoon face as a stand-in")

show(photo, "Working photo" if USING_REAL_PHOTO else "Working photo (cartoon sta
```

Face & eye detection

Same idea as the theory notebook: a **Haar cascade** is a pre-trained XML classifier that slides a window over the image and quickly rejects windows that are not faces. `detectMultiScale` returns a list of `(x, y, w, h)` boxes.

In [ ]:

```
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_fronta
eye_cascade  = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_eye.xm

def detect_face_box(img):
    """Return the biggest detected face as (x, y, w, h), or None if nothing was
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5,
    if len(faces) == 0:
        return None
    # keep the LARGEST box (closest / main face) -> area = w * h
    faces = sorted(faces, key=lambda f: f[2] * f[3], reverse=True)
    return tuple(faces[0])

def detect_eyes(img, face_box):
    """Look for eyes ONLY inside the face box (faster + fewer false positives)."
    x, y, w, h = face_box
    roi_gray = cv2.cvtColor(img[y:y + h, x:x + w], cv2.COLOR_BGR2GRAY)
    eyes = eye_cascade.detectMultiScale(roi_gray, scaleFactor=1.1, minNeighbors=
    # convert eye coords from ROI-local back to full-image coords
    return [(x + ex, y + ey, ew, eh) for (ex, ey, ew, eh) in eyes]

detected = detect_face_box(photo)
print("Detector found a face:", detected is not None, "->", detected)

# FALLBACK: only used for the cartoon stand-in, so every filter below has someth
FACE_BOX = detected if detected is not None else (110, 85, 260, 330)
print("Face box used from here on:", FACE_BOX)

preview = photo.copy()
x, y, w, h = FACE_BOX
cv2.rectangle(preview, (x, y), (x + w, y + h), (0, 255, 0), 3)
for (ex, ey, ew, eh) in detect_eyes(photo, FACE_BOX):
    cv2.rectangle(preview, (ex, ey), (ex + ew, ey + eh), (255, 0, 0), 2)
show(preview, "Detected face (green) and eyes (blue)")
```

✍️ Exercise 1 — Detector settings

Try `minNeighbors` values `1, 3, 5, 10` on your `face.jpg` (skip this exercise if you are using the cartoon stand-in) and print how many faces each setting finds. What happens with very low `minNeighbors`?

In [ ]:
```
# ✍️ Your code here
```

2️⃣ Colour / "lens" filters

These filters change **every pixel** with a formula or a lookup table (LUT) — no face detection needed. This is exactly how Snapchat's non-face "colour" lenses work.

In [ ]:
```
def apply_grayscale(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    return cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)          # back to 3 channels


def apply_invert(img):
    return cv2.bitwise_not(img)


def apply_sepia(img):
    # Classic sepia matrix (applied to RGB order, so we convert first)
    kernel = np.array([[0.272, 0.534, 0.131],
                       [0.349, 0.686, 0.168],
                       [0.393, 0.769, 0.189]])
    rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB).astype(np.float32)
    sepia = rgb @ kernel.T
    sepia = np.clip(sepia, 0, 255).astype(np.uint8)
    return cv2.cvtColor(sepia, cv2.COLOR_RGB2BGR)


def apply_warm(img):
    b, g, r = cv2.split(img)
    r = cv2.add(r, 25)
    b = cv2.subtract(b, 15)
    return cv2.merge([b, g, r])


def apply_cool(img):
    b, g, r = cv2.split(img)
    b = cv2.add(b, 25)
    r = cv2.subtract(r, 15)
    return cv2.merge([b, g, r])


def apply_vignette(img, strength=2.2):
    h, w = img.shape[:2]
    kx = cv2.getGaussianKernel(w, w / strength)
    ky = cv2.getGaussianKernel(h, h / strength)
    mask = ky @ kx.T
    mask = mask / mask.max()                               # 0..1, bright centr
```

```
    out = img.astype(np.float32)
    for c in range(3):
        out[:, :, c] *= mask
    return np.clip(out, 0, 255).astype(np.uint8)


def apply_vintage(img):
    out = apply_sepia(img)
    out = apply_vignette(out, strength=2.0)
    out = cv2.convertScaleAbs(out, alpha=0.9, beta=10)       # slightly faded /
    noise = np.random.default_rng(0).normal(0, 8, out.shape).astype(np.int16)
    out = np.clip(out.astype(np.int16) + noise, 0, 255).astype(np.uint8)
    return out


def apply_color_pop(img, face_box):
    """Grayscale everywhere EXCEPT inside the face box, which stays in colour."""
    gray_3ch = apply_grayscale(img)
    out = gray_3ch.copy()
    x, y, w, h = face_box
    out[y:y + h, x:x + w] = img[y:y + h, x:x + w]
    return out


gallery = [photo, apply_sepia(photo), apply_vintage(photo), apply_warm(photo), a
           apply_vignette(photo), apply_invert(photo), apply_color_pop(photo, FAC
titles = ["Original", "Sepia", "Vintage", "Warm", "Cool", "Vignette", "Invert",
show_many(gallery, titles, cols=4)
```

✍️ Exercise 2 — Your own filter

Write `apply_cyberpunk(img)`: boost blue **and** boost contrast a lot, then add a strong vignette. Combine at least two of the building blocks above (`cv2.convertScaleAbs`, channel `add` / `subtract`, `apply_vignette`).

In [ ]:
```
# ✍️ Your code here
```

3️⃣ Cartoon and beauty filters

In [ ]:
```
def apply_cartoon(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    blur = cv2.medianBlur(gray, 5)
    edges = cv2.adaptiveThreshold(blur, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THR
    color = cv2.bilateralFilter(img, d=9, sigmaColor=200, sigmaSpace=200)
    cartoon = cv2.bitwise_and(color, color, mask=edges)
    return cartoon


def apply_beauty(img, face_box, strength=60):
    """Smooth skin INSIDE the face box only, blended so the edge is not visible.
    x, y, w, h = face_box
    out = img.copy()
    smoothed = cv2.bilateralFilter(img, d=15, sigmaColor=strength, sigmaSpace=st
```

```
    # a soft (feathered) oval mask so the smoothing fades out at the border, not
    mask = np.zeros(img.shape[:2], np.uint8)
    cv2.ellipse(mask, (x + w // 2, y + h // 2), (w // 2, h // 2), 0, 0, 360, 255
    mask = cv2.GaussianBlur(mask, (31, 31), 0).astype(np.float32) / 255.0

    for c in range(3):
        out[:, :, c] = (smoothed[:, :, c] * mask + img[:, :, c] * (1 - mask)).as
    return out


show_many([photo, apply_cartoon(photo), apply_beauty(photo, FACE_BOX)],
          ["Original", "Cartoon filter", "Beauty / skin-smoothing filter"], cols
```

✍️ Exercise 3 — Cartoon strength

Loop over `sigma_values = [50, 150, 300]` for `cv2.bilateralFilter`'s `sigmaColor` / `sigmaSpace` inside a copy of `apply_cartoon`, and show the 3 results side by side. What does a bigger sigma do to the flat colour regions?

In [ ]:
```
# ✍️ Your code here
```

4️⃣ Sticker filters — building transparent (RGBA) assets

Real filter apps glue a **transparent PNG** (RGBA: Red, Green, Blue, **Alpha**) onto the photo. We build our own RGBA "stickers" with OpenCV drawing functions instead of downloading images:

- Draw the sticker's colours on a small canvas.
- Draw the **same shapes in white on a black mask** → that mask becomes the **alpha channel** (255 = fully visible, 0 = fully transparent).
- `overlay_image_alpha()` blends the sticker onto the photo at any (x, y) position, handling the case where it goes past the image edge.

In [ ]:
```
def overlay_image_alpha(background, overlay_rgba, x, y):
    """Paste overlay_rgba (H x W x 4) onto background (H x W x 3) with top-left
    bg_h, bg_w = background.shape[:2]
    ov_h, ov_w = overlay_rgba.shape[:2]

    # clip the overlay so it never goes outside the background
    x1, y1 = max(x, 0), max(y, 0)
    x2, y2 = min(x + ov_w, bg_w), min(y + ov_h, bg_h)
    if x1 >= x2 or y1 >= y2:
        return background                                   # completely off-sc

    ov_x1, ov_y1 = x1 - x, y1 - y
    ov_x2, ov_y2 = ov_x1 + (x2 - x1), ov_y1 + (y2 - y1)

    overlay_crop = overlay_rgba[ov_y1:ov_y2, ov_x1:ov_x2]
    alpha = overlay_crop[:, :, 3:4].astype(np.float32) / 255.0     # shape (h, w
    overlay_rgb = overlay_crop[:, :, :3].astype(np.float32)
```

```
    out = background.copy()
    region = out[y1:y2, x1:x2].astype(np.float32)
    out[y1:y2, x1:x2] = (overlay_rgb * alpha + region * (1 - alpha)).astype(np.u
    return out
```

Sticker 1 — Dog ears + nose + tongue 🐶

In [ ]:

```
def make_dog_ears(w=260, h=180):
    canvas = np.zeros((h, w, 3), np.uint8)
    mask = np.zeros((h, w), np.uint8)
    brown, inner = (19, 69, 139), (100, 149, 237)             # BGR: saddle-brow

    for cx, angle in [(55, 25), (w - 55, -25)]:
        cv2.ellipse(canvas, (cx, 85), (48, 78), angle, 0, 360, brown, -1)
        cv2.ellipse(mask,   (cx, 85), (48, 78), angle, 0, 360, 255, -1)
        cv2.ellipse(canvas, (cx, 95), (24, 45), angle, 0, 360, (180, 105, 255),

    rgba = cv2.cvtColor(canvas, cv2.COLOR_BGR2BGRA)
    rgba[:, :, 3] = mask
    return rgba


def make_dog_nose(w=90, h=110):
    canvas = np.zeros((h, w, 3), np.uint8)
    mask = np.zeros((h, w), np.uint8)
    cv2.ellipse(canvas, (w // 2, 30), (28, 22), 0, 0, 360, (20, 20, 20), -1)
    cv2.ellipse(mask,   (w // 2, 30), (28, 22), 0, 0, 360, 255, -1)
    cv2.circle(canvas, (w // 2 - 9, 24), 4, (90, 90, 90), -1)
    cv2.line(canvas, (w // 2, 48), (w // 2, 65), (20, 20, 20), 4)
    cv2.line(mask,   (w // 2, 48), (w // 2, 65), 255, 4)
    cv2.ellipse(canvas, (w // 2, 78), (26, 16), 0, 0, 180, (20, 20, 20), 4)
    cv2.ellipse(mask,   (w // 2, 78), (26, 16), 0, 0, 180, 255, 4)
    rgba = cv2.cvtColor(canvas, cv2.COLOR_BGR2BGRA)
    rgba[:, :, 3] = mask
    return rgba


def apply_dog_filter(img, face_box):
    x, y, w, h = face_box
    out = img.copy()

    ears = make_dog_ears(int(w * 1.05), int(h * 0.55))
    ears = cv2.resize(ears, (int(w * 1.05), int(h * 0.55)))
    out = overlay_image_alpha(out, ears, x + w // 2 - ears.shape[1] // 2, y - in

    nose = make_dog_nose(int(w * 0.30), int(h * 0.36))
    out = overlay_image_alpha(out, nose, x + w // 2 - nose.shape[1] // 2, y + in
    return out


show(apply_dog_filter(photo, FACE_BOX), "🐶 Dog filter")
```

Sticker 2 — Sunglasses 🕶️

Positioned using the **eyes** if OpenCV found them, otherwise a proportion of the face box (25% down from the top).

In [ ]:
```
def make_sunglasses(w=260, h=90):
    canvas = np.zeros((h, w, 3), np.uint8)
    mask = np.zeros((h, w), np.uint8)
    lens_w, lens_h = int(w * 0.34), int(h * 0.85)
    cy = h // 2

    for cx in [int(w * 0.22), int(w * 0.78)]:
        cv2.ellipse(canvas, (cx, cy), (lens_w // 2, lens_h // 2), 0, 0, 360, (15
        cv2.ellipse(mask,   (cx, cy), (lens_w // 2, lens_h // 2), 0, 0, 360, 255
        cv2.ellipse(canvas, (cx - 12, cy - 12), (14, 7), -25, 0, 360, (150, 150,

    cv2.rectangle(canvas, (int(w * 0.38), cy - 4), (int(w * 0.62), cy + 4), (15,
    cv2.rectangle(mask,   (int(w * 0.38), cy - 4), (int(w * 0.62), cy + 4), 255,
    for cx in [8, w - 8]:
        cv2.line(canvas, (cx, cy), (0 if cx < w // 2 else w, cy - 6), (15, 15, 1
        cv2.line(mask,   (cx, cy), (0 if cx < w // 2 else w, cy - 6), 255, 5)

    rgba = cv2.cvtColor(canvas, cv2.COLOR_BGR2BGRA)
    rgba[:, :, 3] = mask
    return rgba


def apply_sunglasses_filter(img, face_box):
    x, y, w, h = face_box
    eyes = detect_eyes(img, face_box)

    glasses_w = int(w * 0.95)
    glasses = make_sunglasses(glasses_w, int(glasses_w * 90 / 260))

    if len(eyes) >= 2:                                          # use the REAL e
        eyes_sorted = sorted(eyes, key=lambda e: e[0])
        (lx, ly, lw, lh) = eyes_sorted[0]
        (rx, ry, rw, rh) = eyes_sorted[-1]
        centre_y = (ly + lh // 2 + ry + rh // 2) // 2
        place_x = x + w // 2 - glasses.shape[1] // 2
        place_y = centre_y - glasses.shape[0] // 2
    else:                                                        # fall back to
        place_x = x + w // 2 - glasses.shape[1] // 2
        place_y = y + int(h * 0.27)

    return overlay_image_alpha(img.copy(), glasses, place_x, place_y)


show(apply_sunglasses_filter(photo, FACE_BOX), "🕶 Sunglasses filter")
```

Sticker 3, 4, 5 — Moustache, bunny ears, party crown

In [ ]:
```
def make_moustache(w=180, h=70):
    canvas = np.zeros((h, w, 3), np.uint8)
    mask = np.zeros((h, w), np.uint8)
    black = (20, 20, 20)
    cv2.ellipse(canvas, (w // 4, h // 2), (w // 4, h // 3), 0, 0, 180, black, -1
    cv2.ellipse(mask,   (w // 4, h // 2), (w // 4, h // 3), 0, 0, 180, 255, -1)
    cv2.ellipse(canvas, (3 * w // 4, h // 2), (w // 4, h // 3), 0, 0, 180, black
    cv2.ellipse(mask,   (3 * w // 4, h // 2), (w // 4, h // 3), 0, 0, 180, 255,
    cv2.circle(canvas, (w // 2, h // 2 - 5), h // 4, (0, 0, 0), -1)
    cv2.circle(mask,   (w // 2, h // 2 - 5), h // 4, 0, -1)                 # cu
    rgba = cv2.cvtColor(canvas, cv2.COLOR_BGR2BGRA)
```

```
    rgba[:, :, 3] = mask
    return rgba


def make_bunny_ears(w=200, h=260):
    canvas = np.zeros((h, w, 3), np.uint8)
    mask = np.zeros((h, w), np.uint8)
    for cx in [w // 4, 3 * w // 4]:
        cv2.ellipse(canvas, (cx, h // 2 + 20), (30, 120), 0, 0, 360, (235, 235,
        cv2.ellipse(mask,   (cx, h // 2 + 20), (30, 120), 0, 0, 360, 255, -1)
        cv2.ellipse(canvas, (cx, h // 2 + 20), (14, 95), 0, 0, 360, (180, 130, 2
    rgba = cv2.cvtColor(canvas, cv2.COLOR_BGR2BGRA)
    rgba[:, :, 3] = mask
    return rgba


def make_crown(w=260, h=140):
    canvas = np.zeros((h, w, 3), np.uint8)
    mask = np.zeros((h, w), np.uint8)
    gold = (10, 200, 250)
    points = np.array([[10, h - 10], [10, 60], [70, 100], [w // 2, 15],
                       [w - 70, 100], [w - 10, 60], [w - 10, h - 10]])
    cv2.fillPoly(canvas, [points], gold)
    cv2.fillPoly(mask,   [points], 255)
    for cx, col in [(w // 2, (255, 0, 200)), (w // 4, (255, 120, 0)), (3 * w //
        cv2.circle(canvas, (cx, h - 35), 10, col, -1)
        cv2.circle(mask,   (cx, h - 35), 10, 255, -1)
    rgba = cv2.cvtColor(canvas, cv2.COLOR_BGR2BGRA)
    rgba[:, :, 3] = mask
    return rgba


def apply_moustache_filter(img, face_box):
    x, y, w, h = face_box
    m = make_moustache(int(w * 0.5), int(w * 0.5 * 70 / 180))
    return overlay_image_alpha(img.copy(), m, x + w // 2 - m.shape[1] // 2, y +


def apply_bunny_filter(img, face_box):
    x, y, w, h = face_box
    ears = make_bunny_ears(int(w * 0.75), int(h * 0.95))
    return overlay_image_alpha(img.copy(), ears, x + w // 2 - ears.shape[1] // 2


def apply_crown_filter(img, face_box):
    x, y, w, h = face_box
    crown = make_crown(int(w * 1.05), int(h * 0.55))
    return overlay_image_alpha(img.copy(), crown, x + w // 2 - crown.shape[1] //


show_many([apply_moustache_filter(photo, FACE_BOX), apply_bunny_filter(photo, FA
          ["Moustache", "Bunny ears", "Party crown"], cols=3)
```

✍️ Exercise 4 — Design your own sticker

Write `make_party_glasses(w, h)` (a fun coloured-frame glasses RGBA sticker — reuse `make_sunglasses` as a starting point but with a bright colour and thicker "frame"

outline instead of solid black lenses), then write `apply_party_glasses_filter(img, face_box)` using the same positioning idea as `apply_sunglasses_filter`.

In [ ]:
```
# ✍️ Your code here
```

5️⃣ Big-eyes cartoon filter (local warping)

Instead of gluing a sticker, this filter **enlarges the eye regions themselves** — crop a small circle around each eye, resize it bigger, and blend it back centred on the same spot with a soft circular mask.

In [ ]:
```
def apply_big_eyes(img, face_box, scale=1.6):
    out = img.copy()
    eyes = detect_eyes(img, face_box)
    if not eyes:
        print("No eyes detected -> returning the image unchanged (try a clearer
        return out

    for (ex, ey, ew, eh) in eyes:
        cx, cy = ex + ew // 2, ey + eh // 2
        r = int(max(ew, eh) * 0.9)                                  # radius of the

        y1, y2 = max(cy - r, 0), min(cy + r, out.shape[0])
        x1, x2 = max(cx - r, 0), min(cx + r, out.shape[1])
        patch = out[y1:y2, x1:x2]
        if patch.size == 0:
            continue

        big_patch = cv2.resize(patch, None, fx=scale, fy=scale, interpolation=cv
        bh, bw = big_patch.shape[:2]
        bx1, by1 = cx - bw // 2, cy - bh // 2

        # circular soft mask so the enlarged patch blends smoothly, no hard squa
        mask = np.zeros((bh, bw), np.uint8)
        cv2.circle(mask, (bw // 2, bh // 2), int(min(bw, bh) * 0.42), 255, -1)
        mask = cv2.GaussianBlur(mask, (15, 15), 0)
        rgba_patch = cv2.cvtColor(big_patch, cv2.COLOR_BGR2BGRA)
        rgba_patch[:, :, 3] = mask
        out = overlay_image_alpha(out, rgba_patch, bx1, by1)

    return out


show_many([photo, apply_big_eyes(photo, FACE_BOX)], ["Original", "Big-eyes filte
```

✍️ Exercise 5 — Tiny eyes

Copy `apply_big_eyes` into `apply_tiny_eyes(img, face_box, scale=0.6)`. With `scale < 1` the patch shrinks — think about how `bx1, by1` still needs to keep the small patch **centred** on the eye. Test it.

In [ ]:
```
# ✍️ Your code here
```

6️⃣ The filter engine — one function to run them all

A **dictionary of functions** lets us pick a filter by name, exactly like a real app's filter carousel.

In [ ]:
```
FILTERS = {
    "none":        lambda img, box: img,
    "grayscale":   lambda img, box: apply_grayscale(img),
    "sepia":       lambda img, box: apply_sepia(img),
    "vintage":     lambda img, box: apply_vintage(img),
    "vignette":    lambda img, box: apply_vignette(img),
    "color_pop":   lambda img, box: apply_color_pop(img, box),
    "cartoon":     lambda img, box: apply_cartoon(img),
    "beauty":      lambda img, box: apply_beauty(img, box),
    "dog":         lambda img, box: apply_dog_filter(img, box),
    "sunglasses":  lambda img, box: apply_sunglasses_filter(img, box),
    "moustache":   lambda img, box: apply_moustache_filter(img, box),
    "bunny":       lambda img, box: apply_bunny_filter(img, box),
    "crown":       lambda img, box: apply_crown_filter(img, box),
    "big_eyes":    lambda img, box: apply_big_eyes(img, box),
}


def apply_filter(img, name, face_box=None):
    """Apply one named filter. Detects the face automatically if face_box is not
    if name not in FILTERS:
        raise ValueError("Unknown filter: " + name + ". Choices: " + ", ".join(F
    if face_box is None:
        face_box = detect_face_box(img) or FACE_BOX
    return FILTERS[name](img, face_box)


names_to_show = ["dog", "sunglasses", "moustache", "bunny", "crown", "cartoon",
gallery = [apply_filter(photo, n, FACE_BOX) for n in names_to_show]
show_many(gallery, names_to_show, cols=4)
```

✍️ Exercise 6 — Stack two filters

Write `apply_filter_stack(img, names, face_box)` that applies a **list** of filter names **one after another** (loop over the list, feed each output into the next filter). Try `["beauty", "dog"]` and `["vintage", "crown"]`.

In [ ]:
```
# ✍️ Your code here
```

7️⃣ Live webcam filters (run this on your own computer)

`cv2.imshow` opens a real window and needs an actual camera and a display, which this notebook environment does not have. **Copy the function below into a `.py` file** (or a notebook cell if your Jupyter runs locally with a webcam) and run it.

How it works

```
while True:
    read one frame from the webcam
    mirror it (selfie view)
    detect the face (every frame, or every N-th frame for more
speed)
    apply the CURRENTLY SELECTED filter
    draw the face box + FPS counter
    show the frame in a window
    read the keyboard: number keys switch filters, "s" saves a
photo, "q" quits
```

In [ ]:

```python
def live_filter_webcam(camera_index=0, mirror=True, process_every=1):
    """
    Real-time face-filter webcam app.
    Controls: 0=none 1=grayscale 2=sepia 3=vintage 4=cartoon 5=beauty
              6=dog 7=sunglasses 8=moustache 9=bunny 10 not available (use letter
              c = crown, b = big_eyes,  s = save snapshot,  q = quit
    """
    key_to_filter = {
        ord("0"): "none", ord("1"): "grayscale", ord("2"): "sepia", ord("3"): "v
        ord("4"): "cartoon", ord("5"): "beauty", ord("6"): "dog", ord("7"): "sun
        ord("8"): "moustache", ord("9"): "bunny", ord("c"): "crown", ord("b"): "
    }

    cap = cv2.VideoCapture(camera_index)
    if not cap.isOpened():
        print("❌ Could not open the webcam (index", camera_index, "). Check it
        return

    current_filter = "dog"
    last_box = None
    frame_count = 0
    prev_time = cv2.getTickCount()

    print("Live filter app started. Press a number/letter key to change filter,

    while True:
        ok, frame = cap.read()
        if not ok:
            print("⚠️ Could not read a frame - stopping.")
            break

        if mirror:
            frame = cv2.flip(frame, 1)

        frame_count += 1
        if frame_count % process_every == 0 or last_box is None:
            box = detect_face_box(frame)
            if box is not None:
                last_box = box
```

```
        if last_box is not None:
            output = apply_filter(frame, current_filter, last_box)
            x, y, w, h = last_box
            cv2.rectangle(output, (x, y), (x + w, y + h), (0, 255, 0), 2)
        else:
            output = frame

        now = cv2.getTickCount()
        fps = cv2.getTickFrequency() / (now - prev_time)
        prev_time = now
        cv2.putText(output, "FPS: " + str(round(fps, 1)) + "  filter: " + curren
                    (10, 25), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

        cv2.imshow("Snapchat-style filters (press q to quit)", output)

        key = cv2.waitKey(1) & 0xFF
        if key == ord("q"):
            break
        elif key == ord("s"):
            filename = "snapshot_" + current_filter + ".png"
            cv2.imwrite(filename, output)
            print("Saved", filename)
        elif key in key_to_filter:
            current_filter = key_to_filter[key]
            print("Filter switched to:", current_filter)

    cap.release()
    cv2.destroyAllWindows()


# To run it locally:
# live_filter_webcam()
print("Function ready. Call live_filter_webcam() in a LOCAL Python environment w
```

🧪 Proving the pipeline works: a simulated "video" (no camera needed)

We cannot open a webcam here, but we **can** prove the exact same detect → filter → draw loop works on a *sequence of frames* by building a short synthetic video (camera panning across the photo) and running the live-loop logic on it, frame by frame, saving the result as a video file.

In [ ]:
```
fps, out_w, out_h = 15, 400, 400
writer = cv2.VideoWriter("filtered_demo.avi", cv2.VideoWriter_fourcc(*"MJPG"), f

ph, pw = photo.shape[:2]
n_frames = 30
demo_filters_sequence = ["dog", "dog", "sunglasses", "sunglasses", "crown", "cro

saved_preview = []
for i in range(n_frames):
    # a gentle simulated "camera shake" so it behaves like a real, slightly movi
    dx = int(6 * np.sin(i / 4))
    dy = int(4 * np.cos(i / 5))
    M = np.float32([[1, 0, dx], [0, 1, dy]])
    frame = cv2.warpAffine(photo, M, (pw, ph), borderValue=(245, 245, 245))
    frame = cv2.resize(frame, (out_w, out_h))
```

```
    box = detect_face_box(frame) or FACE_BOX                # same fallback log
    filter_name = demo_filters_sequence[i % len(demo_filters_sequence)]
    out_frame = apply_filter(frame, filter_name, box)
    x, y, w, h = box
    cv2.rectangle(out_frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
    cv2.putText(out_frame, filter_name, (10, 25), cv2.FONT_HERSHEY_SIMPLEX, 0.7,

    writer.write(out_frame)
    if i % 6 == 0:
        saved_preview.append(out_frame)

writer.release()
print("Saved filtered_demo.avi with", n_frames, "frames")
show_many(saved_preview, ["frame " + str(i * 6) for i in range(len(saved_preview
```

✍️ Exercise 7 — Your own filter sequence

Build `my_sequence = [...]` with at least 4 filter names from `FILTERS`, and re-run a loop like the one above (you can reuse `photo` / `FACE_BOX`) to create `my_demo.avi`. Cycle through `my_sequence` with the modulo operator `i % len(my_sequence)`, exactly like `demo_filters_sequence` did.

In [ ]:
```
# ✍️ Your code here
```

✅ Solutions

Solution 1 *(needs your own face.jpg)*

In [ ]:
```
if USING_REAL_PHOTO:
    gray_p = cv2.cvtColor(photo, cv2.COLOR_BGR2GRAY)
    for n in [1, 3, 5, 10]:
        found = face_cascade.detectMultiScale(gray_p, scaleFactor=1.1, minNeighb
        print("minNeighbors", n, "-> faces found:", len(found))
    # Very low minNeighbors (e.g. 1) accepts weak matches -> more false positive
else:
    print("Using the cartoon stand-in - upload face.jpg to try this exercise for
```

Solution 2

In [ ]:
```
def apply_cyberpunk(img):
    b, g, r = cv2.split(img)
    b = cv2.add(b, 40)
    out = cv2.merge([b, g, r])
    out = cv2.convertScaleAbs(out, alpha=1.4, beta=-30)
    out = apply_vignette(out, strength=1.8)
    return out

show(apply_cyberpunk(photo), "Cyberpunk filter")
```

Solution 3

In [ ]:
```
def cartoon_with_sigma(img, sigma):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    blur = cv2.medianBlur(gray, 5)
    edges = cv2.adaptiveThreshold(blur, 255, cv2.ADAPTIVE_THRESH_MEAN_C, cv2.THR
    color = cv2.bilateralFilter(img, d=9, sigmaColor=sigma, sigmaSpace=sigma)
    return cv2.bitwise_and(color, color, mask=edges)

results = [cartoon_with_sigma(photo, s) for s in [50, 150, 300]]
show_many(results, ["sigma=50", "sigma=150", "sigma=300"], cols=3)
# Bigger sigma -> flat colour regions merge together more (more "painted" / post
```

Solution 4

In [ ]:
```
def make_party_glasses(w=260, h=90):
    canvas = np.zeros((h, w, 3), np.uint8)
    mask = np.zeros((h, w), np.uint8)
    frame_color = (255, 60, 200)
    lens_w, lens_h = int(w * 0.34), int(h * 0.85)
    cy = h // 2
    for cx in [int(w * 0.22), int(w * 0.78)]:
        cv2.ellipse(canvas, (cx, cy), (lens_w // 2, lens_h // 2), 0, 0, 360, fra
        cv2.ellipse(mask,   (cx, cy), (lens_w // 2, lens_h // 2), 0, 0, 360, 255
    cv2.rectangle(canvas, (int(w * 0.38), cy - 4), (int(w * 0.62), cy + 4), fram
    cv2.rectangle(mask,   (int(w * 0.38), cy - 4), (int(w * 0.62), cy + 4), 255,
    rgba = cv2.cvtColor(canvas, cv2.COLOR_BGR2BGRA)
    rgba[:, :, 3] = mask
    return rgba

def apply_party_glasses_filter(img, face_box):
    x, y, w, h = face_box
    glasses = make_party_glasses(int(w * 0.95), int(w * 0.95 * 90 / 260))
    return overlay_image_alpha(img.copy(), glasses, x + w // 2 - glasses.shape[1

show(apply_party_glasses_filter(photo, FACE_BOX), "Party glasses")
```

Solution 5

In [ ]:
```
def apply_tiny_eyes(img, face_box, scale=0.6):
    out = img.copy()
    eyes = detect_eyes(img, face_box)
    for (ex, ey, ew, eh) in eyes:
        cx, cy = ex + ew // 2, ey + eh // 2
        r = int(max(ew, eh) * 0.9)
        y1, y2 = max(cy - r, 0), min(cy + r, out.shape[0])
        x1, x2 = max(cx - r, 0), min(cx + r, out.shape[1])
        patch = out[y1:y2, x1:x2]
        if patch.size == 0:
            continue
        small_patch = cv2.resize(patch, None, fx=scale, fy=scale)
        sh, sw = small_patch.shape[:2]
        sx1, sy1 = cx - sw // 2, cy - sh // 2
        mask = np.full((sh, sw), 255, np.uint8)
        rgba_patch = cv2.cvtColor(small_patch, cv2.COLOR_BGR2BGRA)
        rgba_patch[:, :, 3] = mask
        out = overlay_image_alpha(out, rgba_patch, sx1, sy1)
    return out
```

```
show(apply_tiny_eyes(photo, FACE_BOX), "Tiny eyes filter")
```

Solution 6

In [ ]:
```
def apply_filter_stack(img, names, face_box):
    out = img
    for name in names:
        out = apply_filter(out, name, face_box)
    return out

show_many([apply_filter_stack(photo, ["beauty", "dog"], FACE_BOX),
           apply_filter_stack(photo, ["vintage", "crown"], FACE_BOX)],
          ["beauty + dog", "vintage + crown"], cols=2)
```

Solution 7

In [ ]:
```
my_sequence = ["moustache", "crown", "sepia", "big_eyes"]
writer2 = cv2.VideoWriter("my_demo.avi", cv2.VideoWriter_fourcc(*"MJPG"), 15, (4

for i in range(20):
    dx = int(5 * np.sin(i / 3))
    M = np.float32([[1, 0, dx], [0, 1, 0]])
    frame = cv2.resize(cv2.warpAffine(photo, M, (pw, ph), borderValue=(245, 245,
    box = detect_face_box(frame) or FACE_BOX
    name = my_sequence[i % len(my_sequence)]
    out_frame = apply_filter(frame, name, box)
    cv2.putText(out_frame, name, (10, 25), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 25
    writer2.write(out_frame)

writer2.release()
print("Saved my_demo.avi")
```

🚀 Next steps

- Swap the manual-proportion sticker placement for real **facial landmarks** (`pip install mediapipe` , `FaceMesh`) — filters will track eyes/nose precisely even when you tilt your head.
- Add **face tracking between frames** (e.g. a simple centroid tracker) so the box does not need to be re-detected every single frame — smoother and faster.
- Package `live_filter_webcam()` into a small script with `argparse` so you can launch it as `python filters.py --filter dog` from the terminal.
- Try building an **Instagram-style boomerang**: record a short clip, play it forward then backward using `moving_ball` -style frame lists.

🎉 You now have your own mini Snapchat filter engine, built entirely with OpenCV!